A Tour of AI–Assisted Development

AI Coding Agent Workshop — Beginner Session (Codex)

Goal: Get a feel for the AI–assisted development landscape and complete your first successful AI collaboration

John Wu | Apr 2026

Today's journey

Segment	What we'll do
Concepts	Build up the core ideas
Live demo	Watch AI work step by step
Hands-on practice	Apply it to your own work
Recap & Q&A	Key takeaways + homework

> The focus today is to **experience the full workflow end-to-end**, not to memorize every term

Part 1: Concepts

What is Vibe Coding?

The traditional way

You → think through logic → write code → debug → done

Vibe Coding

You → **describe intent** → AI generates → **you verify** → done

> Instead of writing line by line, you use natural language to "say what you want"

Three mental model shifts

Before	Now
Search — Google, Stack Overflow	**Conversation** — ask AI directly, get full answers
Copy-paste — find code, tweak until it works	**Describe intent** — say what you want, AI tailors it
Solo dev — think and write it all yourself	**Human-AI collaboration** — you decide, AI executes

AI tools

	Claude Code	Codex	Gemini CLI
Company	Anthropic	OpenAI	Google
Project instructions	CLAUDE.md	AGENTS.md	GEMINI.md
Extensibility	Skills / MCP	Skills / MCP	Extensions / MCP
Common ideas	All support project instructions and can connect to external tools and data sources		

> The beginner session uses **Codex** as the main tool

Prompts — how you ask in the moment

In plain terms

Every sentence you say to AI is a Prompt

Example

"Add an export feature to `src/bookmark.ts` for me"

> The more specific you are, the closer the result will be to what you want

Project Instructions — give AI context on your project

In plain terms

Drop a project instruction file into your repo. AI reads it automatically, so you don't need to repeat yourself every time.

Example

Inside `AGENTS.md` (for Codex), you write:
"This is a TypeScript project. We use Vitest for testing. Commit format is type(scope): description."

> Write it once, and the whole team benefits — AI knows the rules every time it shows up

Skills — teach AI to follow a workflow

In plain terms

Package common workflows into reusable capabilities that AI can apply when needed

Example

Once you've written a "TDD skill":

- You can explicitly tell AI to use it
- Or let AI apply it automatically when the task matches

> Write once, reuse forever — no need to re-explain every time

MCP — connect AI to external tools and data

In plain terms

Let AI go beyond what you paste in — it can reach external systems to query data, read documents, and take actions

Example

- Connect to GitLab → read Issues and MRs
- Connect to JIRA → read tickets
- Connect to Telegram → send and receive messages

> AI evolves from "just chatting" to "querying, reading, and actually doing things"

The "Opening a Restaurant" analogy

What you say	Restaurant analogy	Concept
"I want something spicy"	Cook a first draft and iterate	**Vibe Coding**
"Kung Pao chicken, no peanuts, less oil"	The more specific, the better the dish	**Prompts**
"This is a Sichuan restaurant, high-heat wok"	Kitchen rules are written down	**Project Instructions**
Recipe + checkpoints + tool list	Standard operating procedure	**Skill**
Connect to suppliers, warehouse, delivery	External interfaces	**MCP**

Context — how does all this actually work?

Every time you talk to AI, the tool typically assembles relevant **Context** for it, such as:

```
your prompt + conversation history + relevant file contents + project instructions (e.g. AGENTS.md) + Skills loaded on demand + results from MCP calls
```

The cost

- Bigger context → usually slower and more expensive
- Longer conversations → earlier key points get diluted
- Too much irrelevant info → judgment quality drops

> So: stay on point, start a new conversation when needed, don't dump everything in

Models — the AI's spell power

Think of different models as different spell tiers in a game:

Tier	Feel	Cost	When to use
Basic	Fastest, cheapest	🔥	Copy editing, formatting, simple Q&A
Workhorse	Balanced speed and quality	🔥🔥	Everyday feature work, most tasks
Ultimate	More reliable on hard stuff	🔥🔥🔥	Stubborn bugs, architecture, heavy refactors

> Model names differ across tools, but the principle is the same:
> **don't use a sledgehammer for a nail**

Reasoning effort — how long the spell charges

The same spell can be cast with different charge times:

Charge	Feel	Cost	When to use
Low	Near-instant	🔥	Trivial tweaks, formatting
Medium	Thinks briefly	🔥🔥	General development tasks
High	Thinks longer, hits harder	🔥🔥🔥	Complex logic, multi-step problems
Max	Full charge before the big hit	🔥🔥🔥🔥	Hard debugging, architecture, tough breakdowns

> Longer charging usually means more complete answers, but it's also slower and pricier. You don't always need to max it out.

Token — the AI's mana

Think of **Token** as the unit AI uses to measure information — or "mana spent" for short.

- The more you say, the more it costs
- The more AI replies, the more it costs
- Longer, messier conversations usually cost more

> So: stay on point, cut the fluff, and start a new chat when stuck

What if you run out of mana?

- Different tools and plans have different limits
- Some count tokens, others count messages, quotas, or rate limits
- Some plans reset periodically
- Some let you upgrade or top up — but not all
- Stronger models, long conversations, and heavy tool use all burn more

> So: simple tasks get basic spells, save the ultimates for hard problems
> For exact limits and resets, check each tool's official plan details

At the beginner level, just remember three things

1. State what you need clearly
2. Let AI do the work
3. Always verify the result

> Success today isn't knowing every tool — it's completing one verifiable collaboration

Part 2: Live demo

A tour of the Codex UI

Demo 1 — Vibe Coding

Build a game with one sentence

1. Tell Codex: **"Build me a whack-a-mole game"**
2. After the first version, iterate with feedback:
   - "Add an explosion effect when a mole gets hit"
   - "Make the moles appear faster over time"
3. Ship it once you're happy

Demo 2 — The power of Prompts

Same need, different phrasings

	Prompt	Expected result
❌	"Make it better"	AI has no idea what to change
⚠️	"The UI is ugly, fix it"	AI guesses, probably wrong
✅	"Change the background to a grass theme, make the moles cartoon-style, add rounded corners and shadows to buttons, add a hit effect, and turn the cursor into a hammer"	AI executes precisely

Demo 3 — AGENTS.md

Make AI remember your project's rules

Continuing from Demo 1, create `AGENTS.md`:

```
# Whack-a-Mole Game

## Tech stack
HTML + CSS + vanilla JavaScript, no frameworks

## Rules
- All text in Traditional Chinese
- CSS uses a cartoon style, rounded corners + bright colors
- Every change must be testable by opening index.html directly
```

> From now on, Codex will usually read these rules first and try to follow them.

Demo 4 — Skills

Teach AI to follow a workflow

```
In the project, create a `code-review` Skill.
Every time you run the final check, execute:
1. Check for leftover console.log calls
2. Confirm all text is in Traditional Chinese
3. Verify the game opens correctly
4. List the files changed this round
```

> Later, you can call this skill explicitly, or let AI apply it when the task fits

Demo 5 — Superpowers

A full skill pack

1. Install Superpowers
2. Describe your task to Codex, and specify a skill when it matters
   e.g. "Start with brainstorming to refine the requirements, then add a real-time multiplayer mode"
3. Watch how it breaks the work into a more structured flow

What to notice

- No need to rewrite common workflows yourself
- AI works more consistently using the installed skill pack

Demo 6 — OpenSpec

Spec-driven development

1. Install OpenSpec, run `openspec init`
2. `/opsx:propose` — create a requirement proposal (e.g. "`/opsx:propose` add difficulty selection to the whack-a-mole game")
3. `/opsx:apply` — once the spec is approved, let AI implement it
4. `/opsx:archive` — after acceptance, archive it

> Requirements are recorded, changes are tracked, and AI follows the spec instead of fuzzy memory

Demo recap — from loose requests to tracked work

Demo	Topic	What you learned
1	Vibe Coding	Ship it with one sentence
2	Prompts	Be clear, get better results
3	AGENTS.md	Write rules once, apply forever
4	Skills	Automate your workflows
5	Superpowers	One-click install for a skill pack
6	OpenSpec	Turn requirements into tracked docs

Part 3: Hands-on practice (20 min)

Apply today's methods to your own work

⚠️ Before you start: don't paste sensitive data

- **Don't paste** API keys, passwords, tokens, or customer PII
- **Don't paste** unreleased core algorithms or business strategy
- Not sure if it's safe? → Practice with public or mocked data first

> Details are in the recap — for now, just remember: **when in doubt, don't paste**

Free practice

Now it's your turn — use what you just saw on your own work

- Don't aim big — just finish one small, verifiable result
- The goal isn't to get it right on the first try; it's to let AI draft, then iterate
- Today's bar: **you know how to start, how to follow up, and how to verify**

How to pick a practice topic

Three criteria

- **Small** — a single feature, bug, or cleanup task, not cross-system
- **Clear** — you can say what you want in one sentence
- **Verifiable** — you can judge the result with a test, screen, or checklist

Topics to avoid

- "Refactor the whole backend"
- "Make the entire system prettier"
- "Build a new product end-to-end"

Not sure how to open? Use this template

Engineers

"Here's my small task: ____. Please write the first version, modify the code directly if needed, and at the end tell me how to verify it, or run the relevant tests for me."

QA

"Here's the feature I'm testing / the issue I observed: ____. Please turn it into a test checklist or bug report in a clear, directly usable format."

BA

"Here's my requirement description: ____. Please turn it into user stories and acceptance criteria, and point out where it's still unclear."

When you get stuck, follow up like this

Common follow-ups

- "The test is failing with this error, please fix it until it passes: ____"
- "The result differs from what I expected. The difference is: ____"
- "Don't do it all at once — break it into 3 steps"
- "Tell me how you plan to change it before you start"

Done criteria

- [ ] AI produced the first version for you
- [ ] You gave at least one round of feedback or direction
- [ ] You have a real basis for acceptance, not just a vibe

Part 4: Recap

The full map we walked today

```
✅ Vibe Coding   — Ship first, refine later
✅ Prompts       — The clearer you ask, the closer the answer
✅ AGENTS.md     — Tell AI your project's rules
✅ Skills        — Let AI follow a fixed workflow
✅ Superpowers   — Package common flows as reusable capabilities
✅ OpenSpec      — Turn requirements into tracked specs
```

Safety reminder: AI is not a safe

What NOT to paste into AI

Type	Example	Risk
Confidential code	Core algorithms, unreleased features	Sensitive info sent to third parties
Customer data	PII, transaction records	Privacy law violations
Credentials	API keys, passwords, tokens	Immediately exploitable if leaked
Internal documents	Unpublished business strategy	Competitive info leaked

> Whether it's usable depends on company policy, tool settings, and data classification rules

After class, here's how to start

Do these three things when you get back

1. Pick a small task you can **verify within 30 minutes**
2. Let AI do the first version — don't chase perfection up front
3. Feed errors, differences, and constraints back in to iterate

Three takeaways

1. AI is your pair programmer

AI is a collaborator, not a replacement — **you own the review**

2. AI isn't just about Prompts

Longer isn't better — clearer is. Lead with **goal, constraints, acceptance**

3. AI makes mistakes

When stuck, don't force-fix. Send the **error and the expected diff** back to AI

Q&A